

7-5 · Hybrid Search 구현 · 실습 1.5H

Hybrid 결합과 주차 산출물 완성

RRF 열 줄, 답변 체인, 그리고 산출물 두 가지

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① `rrf()` 를 작성하고 이론 시간의 예시로 `assert` 검증한다
- ② `hybrid_search` 로 세 방식을 나란히 비교하고 `k` 를 바꿔 본다
- ③ 컨텍스트 밖 질문에 「모른다」고 답하는 체인과 출처 표시를 붙인다
- ④ `vector-schema.yaml` · `build_kb.py` 를 검증하고 제출한다

제출 전 확인 — 두 번 실행해도 총 개수가 같은가 · `size` 가 실제와 같은가 · API 키가 코드에 없는가

아홉 단계 — 결합 · 체인 · 산출물 · 제출

1H 실습의 `bm25_rank · vector_rank · text_by_id · records` 를 그대로 이어서 씁니다

1·2. RRF

`rrf()` →
`hybrid_search`

3·4. 비교

`compare()` ·
k 실험

5·6. 체인

`ask()` ·
`ask_with_source`

7·8. 산출물

`schema.yaml` ·
`build_kb.py`

9. 제출

두 번 실행 ·
`grep sk-`

핵심 코드 — 결합과 체인 다섯 줄

RRF 한 줄	<code>scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank + 1)</code>
검증	<code>assert rrf([["A","B","C"], ["B","D","A"]])[0] == "B"</code>
결합	<code>fused = rrf([bm25_rank(query), vector_rank(query)])</code>
체인	<code>chain = prompt llm StrOutputParser()</code>
호출	<code>chain.invoke({"context": context, "question": question})</code>

assert 가 통과해야 실제 데이터에 쓴다 — 순서를 바꾸면 틀려도 모른다

합치고, 세 방식을 나란히 놓고, k 를 혼든다

1 **hybrid_search(query, top_k=3)**
양쪽 상위였던 조각이 맨 위 · 한쪽에만 있던 문서도 뒤에 남는다

2 **compare() — ERR-4021 · 풀어 쓴 질문 · Kafka 파티션**
하이브리드가 좋은 쪽을 따라가는가 — 손해 보는 질문도 적어 둔다

3 **k = 1 · 5 · 60 · 200**
작을수록 각 검색기 1등이 강하게, 커지면 후보가 고르게 섞인다

기본값

k = 60

무난한 지점

id 타입을 섞으면 같은 문서가 합쳐지지 않는다

vector_rank 의 str(h.id) 에서 str 을 잠시 빼고 rrf 를 돌려 보세요 — 같은 문서가 다른 키로 집계됩니다

str 빼기	rrf 결합	증상	되돌리기
[h.id for h in hits] int · UUID 객체	같은 문서가 두 키로	결과가 두 배 · 합쳐지지 않음	str(h.id) 로 통일 → 정상

답변 체인 — 근거 안에서만 답하고 출처를 붙인다

- 1 prompt | llm | StrOutputParser()**
system: 아래 컨텍스트만 근거로 · 없는 내용이면 모른다고 답해줘
- 2 ask(question)**
hybrid_search 결과를 --- 로 이어 {context} 에 넣는다
- 3 두 질문으로 확인**
「도구가 뭐야?」는 근거로 답 · 「김치찌개 끓이는 법」은 모른다
- 4 ask_with_source**
payload 의 source 집합을 정렬해 답변 아래 [근거] 로 덧붙인다
- 5 RAG 의 가장 큰 가치**
사람이 답을 검증할 수 있게 — 7-2 의 메타데이터가 여기서 값을 한다

vector-schema.yaml — 정의서를 코드로 대조

항목	값	비고
collection	lecture_kb	7-4 와 같은 이름
embedding.model	text-embedding-3-small	또는 nomic-embed-text
vector.size · distance	1536 · Cosine	nomic 은 768 — 반드시 일치
payload	text · source · chunk_index	string · string · integer
chunking	chunk_size 500 · overlap 50	8단계 스크립트와 같은 값
id_rule	uuid5(source#chunk_index)	재실행 안전(먹등)

splitter: RecursiveCharacterTextSplitter · 대조: assert cfg.size == spec["vector"]["size"] — 문서와 실체가 어긋나는 것이 가장 흔한 사고

build_kb.py — 두 번 돌리고 제출

MinIO 의 .txt 를 읽어 Qdrant 에 색인 — delete → upsert 와 uuid5 id 로 몇 번 실행해도 결과가 같다

1회	2회	CHUNK_SIZE=300	grep -rn "sk-" .	제출
python build_kb.py tail -1	총 개수가 같으면 합격	개수는 달라지되 오류 없이	API 키가 섞이지 않았는가	yaml · py · 비교 기록(권장)

이 1.5시간이 끝날 때 확인할 것

- 1 `assert rrf(...)[0] == "B"` 가 통과한다
- 2 하이브리드 상위에 양쪽 상위였던 조각, 뒤에 한쪽 문서도 남는다
- 3 `k` 를 바꾸면 상위 2개가 달라지는 것을 봤다
- 4 지식베이스 밖 질문에 '모른다'고 답하고 [근거] 가 붙는다
- 5 정의서의 `size` 가 실제 컬렉션과 같다 (`assert` 통과)
- 6 `build_kb.py` 두 번의 총 개수가 같고 `grep "sk-"` 에 아무것도 없다

7-5 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

7주차 닷새를 한 표로 잇고, 자주 겪는 오류 네 가지를 짚은 뒤 8주차 Airflow 를 준비합니다

✓ 7-1~7-5 한 일과 남은 결과물

✓ 체크리스트로 오늘 실습 점검

✓ id 타입 · 조사 · 컬렉션 이름 · 재색인 오류

✓ docker compose version — 8주차 준비

가져갈 것

제출 완료 · 검색 비교 기록